FIAP

Data Structure and Algorithms

Ciência da Computação

Prof. Érick T. Yamamoto

Syllabus – Decisões e Laços: Toma de Decisão e Repetição

O que vocês irão aprender nesta aula:

- Entender o conceito de decisão:
  - Aprender a estrutura IF;
  - Utilizar operadores relacionais;
  - Construir lógica de decisão.
- Entender repetição de código:
  - Utilizar for, while e do-while;
  - Controlar iterações;
  - Evitar repetição manual de instruções.

Problema!

Um programa precisa tomar decisões:

Se o aluno tem média >= 6 → aprovado

Se não → reprovado

Como fazemos isso em C?

Decisão!

O que é uma decisão?

O que é decisão?

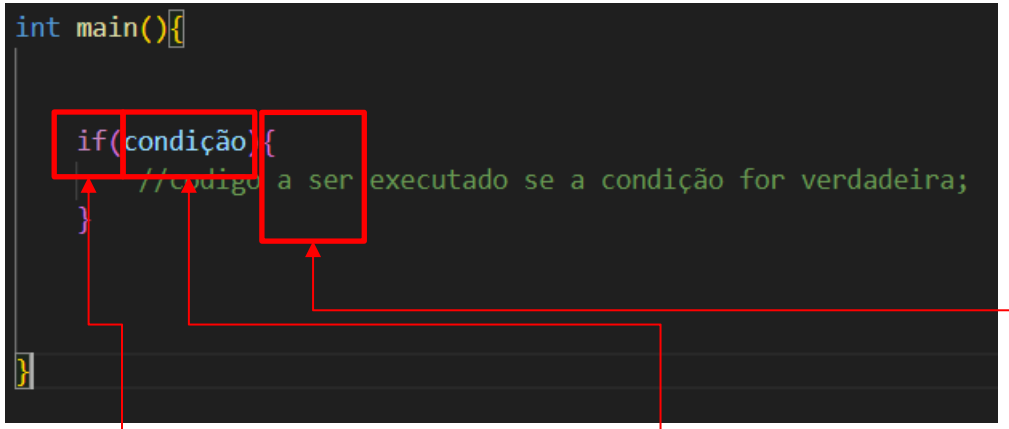
Programas não fazem só cálculos.

Eles também tomam decisões com base em condições.

Exemplo:

- idade >= 18 → pode dirigir

Estrutura básica do IF:



Parâmetro

A condição a ser verificada

Entre {.} é a lógica a ser executada se a condição for verdadeira.

É importante lembrar que nesta estrutura, não há necessidade do ;

Vamos colocar a mão na massa!

Dado o exemplo, vamos construir a condição:

- idade >= 18 → pode dirigir

```c
#include<stdio.h>

int main() {
    int idade = 18;

    if (idade >= 18) {
        printf("Maior de idade\n");
    }

    return 0;
}
```

No terminal deve aparecer:

```
Maior de idade
```

Como o IF funciona

Vamos pegar isoladamente a estrutura **IF** do exemplo anterior, temos:

```c
if (idade >= 18) {
    printf("Maior de idade\n");
}
```

Você pode perceber que a condição utiliza operações relacionais, então é importante o relembrar os simbolos > < >= <= == !=

1. **Avalia a condição**
2. **Se for verdadeira → executa**
3. **Se for falsa → ignora**

Como o IF funciona com + condições

Existe uma outra maneira de estruturar as condições de forma mais afunilada.

Utilizamos para isso o ELSE e ELSE IF.

```
if (media >= 9) {
    printf("Excelente\n");
    } else if (media >= 6) {
        printf("Aprovado\n");
    } else {
        printf("Reprovado\n");
    }
```

1. **Avalia a condição**
2. **Se for verdadeira → executa**
3. **Se for falsa → verifica a próxima**
4. **Se não encontrar → ignora**

Como o IF funciona com + condições

Se você quiser ter mais condições e restrições para maior precisão da decisão, em C com operadores lógicos **“E”** e **“OU”.** Representada, respectivamente, como **&&** e **||**.

```c
if (media >= 9 && falta >= 75 || recuperacao >= 7) {
```

```c
int main() {
    int media;
    int falta;
    if (media >= 9 && falta >= 75){
        printf("Excelente e Aprovado!\n");
        } else if (media >= 6 && falta >= 75) {
            printf("Aprovado\n");
        } else if (media < 6 && falta >= 75) {
            printf("Reprovado, media insuficiente\n");
        } else {
            printf("Reprovado, falta insuficiente\n");
        }


    return 0;
}
```

Não esquecer da ordem de precedência aqui também

Você pode verificar que existem duas condições para entrar que é através da media e falta. Definida pela lógica “E”.

Laços de Repetição em C Automatizando tarefas no programa

Problema!

Se eu quiser imprimir 10 números...

```
print("1\n");
print("2\n");
print("3\n");
print("4\n");
print("5\n");
print("6\n");
```



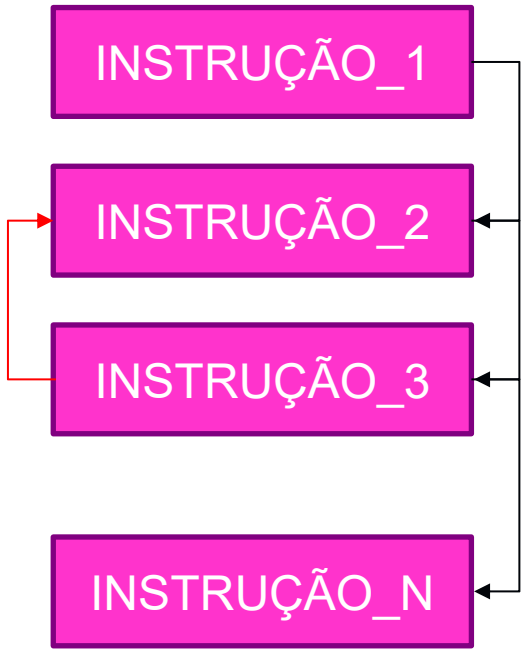
O que é e Como funciona as repetições?

Laço = repetir um bloco de código várias vezes

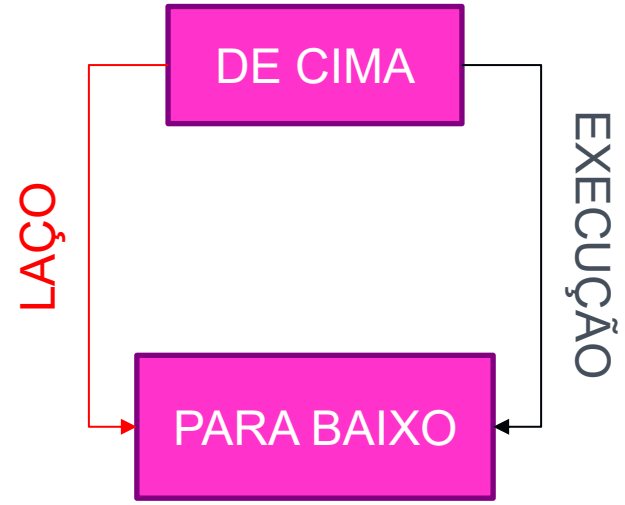
TIPOS:

1. **for**
2. **while**
3. **do-while**

ESTRUTURA



LEGENDA:



LAÇO DE REPETIÇÃO - LOOP FOR

O **for** foi criado para facilitar loops com controle completo.

Muito usado em:

- linguagens científicas;
- FORTRAN (1950s);
- depois adaptado em C.

👉 **O nome vem de → FOR = para cada valor**

A estrutura fica:

```
for(inicio; condicao; incremento) {
    // repetição controlada
}
```

Exemplo:

```c
#include <stdio.h>

int main() {

    for(int i = 0; i < 5; i++) {
        printf("%d\n", i);
    }

    return 0;
}
```

LAÇO DE REPETIÇÃO - WHILE

O **while** vem das primeiras linguagens estruturadas, como:

- ALGOL (década de 1950–60);
- posteriormente C (Dennis Ritchie, 1972).

👉 Era a forma mais simples de representar repetição baseada em condição.

Ideia original: “repita enquanto isso for verdadeiro”

A estrutura fica:

```
while(condicao) {
    // repete enquanto for verdadeiro
}
```

Exemplo:

```
#include <stdio.h>

int main() {
    int i = 0;

    while(i < 5) {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

LAÇO DE REPETIÇÃO - DO-WHILE

Também herdado de linguagens como:

- ALGOL;
- Pascal;
- depois incorporado em C.

A estrutura fica:

```
do {
    // executa primeiro
} while(condicao);
```

👉 **Foi criado para resolver um problema: "E se eu quiser garantir que o código execute pelo menos uma vez?"**

Exemplo:

```
#include <stdio.h>

int main() {
    int i = 0;

    do {
        printf("%d\n", i);
        i++;
    } while(i < 5);

    return 0;
}
```

LAÇO DE REPETIÇÃO – WHILE vs DO-WHILE

```
while(condicao) {
    // repete enquanto for verdadeiro
}
```

```
do {
    // executa primeiro
} while(condicao);
```

Quando executa

TESTA ANTES

EXECUTA ANTES

O do-while garante: “faça pelo menos uma vez, depois decide se continua”.

LAÇO DE REPETIÇÃO - NÃO CONVENCIONAL

Antes das estruturas modernas, os programas usavam → **goto**

👉 Isso vem de linguagens muito antigas (anos 50–60)

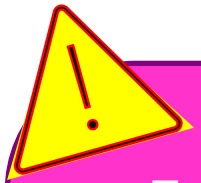
```c
#include <stdio.h>

int main() {
    int i = 0;

inicio:
    if(i >= 5) return 0;

    printf("%d\n", i);
    i++;

    goto inicio;
}
```



Esse foi um dos motivos da criação de estruturas como for e while.

👉 **Código com goto ficava:**

1. **difícil de ler;**
2. **difícil de manter;**
3. **cheio de erros.**

LAÇO DE REPETIÇÃO - NÃO CONVENCIONAL

Recursão vem da matemática:

- estudada muito antes da computação;
- formalizada em lógica matemática.

Na computação:

- usada em linguagens como Lisp;
- depois em C.



Recursão é uma forma alternativa de repetição, baseada em chamadas de função.

```c
#include <stdio.h>

void contar(int i) {
    if(i > 5) return;

    printf("%d\n", i);
    contar(i + 1);
}

int main() {
    contar(1);
    return 0;
}
```

Função que chama ela mesma.

VAMOS COLOCAR NA PRÁTICA!

ATIVIDADE 1 — FOR (controle conhecido)

📌 Enunciado

Crie um programa que:

- imprima os números de 1 a 10;
- imprima também a soma desses números.

ATIVIDADE 2 — WHILE (condição)

📌 **Enunciado**

Crie um programa que:

- leia números do usuário;
- continue lendo enquanto o número for diferente de **0**;
- ao final, mostre a **soma dos números digitados.**

VAMOS COLOCAR NA PRÁTICA!

ATIVIDADE 3 — DO-WHILE (executa pelo menos uma vez)

📌 **Enunciado**

Crie um programa que:

- peça ao usuário um número;
- repita a leitura até que ele digite um número **positivo;**
- ao final, exiba o número digitado.

ATIVIDADE 4 — ENCONTRAR O ERRO (FOR)

📌 **Enunciado**

O código abaixo deveria imprimir números de 1 a 5, mas está errado. Identifique e corrija o erro.

```c
#include <stdio.h>

int main() {

    for(int i = 1; i <= 5; i--) {
        printf("%d\n", i);
    }

    return 0;
}
```

VAMOS COLOCAR NA PRÁTICA!

ATIVIDADE 5 — ENCONTRAR O ERRO (WHILE)

📌 Enunciado

O código abaixo deveria somar números de 1 a 5, mas não funciona corretamente. Identifique e corrija o erro.

```c
#include <stdio.h>

int main() {

    int i = 1;
    int soma = 0;

    while(i <= 5) {
        soma += i;
    }

    printf("Soma = %d\n", soma);

    return 0;
}
```


DÚVIDAS?!
OBRIGADO!!

Copyright © 2026
Prof. Dr. Erick T. Yamamoto.
Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor (autor).